COMPSCI 308 Design and Analysis of Algorithms

Quiz 02

Xing Shi Cai

2026-02-02

👀 Don't Cheat

During the quiz/exam do *not*

- 🩹 give or receive aid;
- ⏱️ read the questions before the quiz begins;
- 🛑 keep writing when the quiz ends;
- 📱 use a phone, calculator, computer or the internet;
- 🚽 hide textbooks or other aids in the toilet.

⚡ Follow Rules

- 👨‍🎓 Enter your name and **SID** (not your Net ID) in the designated space below.
- 🆎 Support your proofs with both equations and descriptive reasoning.
- ✏️ Use only pen for writing as pencil marks are not clearly visible.
- ❓ Restrict your inquiries to clarifications regarding typos.

Name: SID:

Figure 1: Good Luck

1 Greedy Algorithm (1pt)

The standard greedy algorithm for the Activity Selection Problem selects the activity with the *earliest finish time* compatible with those already selected. Suppose that instead, we select the activity with the **latest start time** that is compatible with all previously selected activities.

1.1 (0.4pt)

Run this algorithm on the following set of activities. List the indices of the selected activities in increasing order.

- Activity 1: $[1, 4)$
- Activity 2: $[3, 5)$
- Activity 3: $[0, 6)$
- Activity 4: $[5, 7)$
- Activity 5: $[3, 8)$
- Activity 6: $[6, 10)$
- Activity 7: $[8, 11)$
- Activity 8: $[2, 14)$

Answer:

$$\boxed{2, 4, 7}$$

1.2 (0.2pt)

Explain in one sentence why this strategy is considered a **greedy** strategy.

💡 Focus on the local choice property of a greedy algorithm.

Answer:

This is Exercise 15.1-2. Please see the solution manual for the correct answer.

1.3 (0.4pt)

Prove that the algorithm yields an optimal solution.

💡 Hint: Consider the symmetry between this problem and the standard activity selection problem. One sentence is sufficient.

Answer:

This is Exercise 15.1-2. Please see the solution manual for the correct answer.

2 Huffman Code — Lemma 15.3 (1pt)

👀 Lemma 15.2

For the least frequent characters $x, y \in C$, there exists an optimal prefix code where codewords for x and y have equal length and differ only in the last bit.

Lemma 15.3

Let $x, y \in C$ be the characters with minimum frequencies.
Let $C' = C - \{x, y\} \cup \{z\}$ where $z.freq = x.freq + y.freq$.
For a tree T' representing an optimal prefix code for C', the tree T formed by replacing T''s leaf node for z with an internal node having x and y as children, yields an optimal prefix code for C.

We will prove Lemma 15.3 by following the steps below.

2.1 (0.2pt)

Cost Relationship: Show that $B(T) = B(T') + x.freq + y.freq$.

Answer:

For each $c \in C - \{x, y\}$, we have $d_T(c) = d_{T'}(c)$, so $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$.
Since $d_T(x) = d_T(y) = d_{T'}(z) + 1$, we get

$$\begin{aligned} & x.freq \cdot d_T(x) + y.freq \cdot d_T(y) \\ & = (x.freq + y.freq)(d_{T'}(z) + 1) \\ & = z.freq \cdot d_{T'}(z) + (x.freq + y.freq). \end{aligned}$$

Summing over all characters,

$$B(T) = B(T') + x.freq + y.freq.$$

2.2 (0.2pt)

Contradiction Assumption: To prove by contradiction, what assumption should we make about T? What does this imply about the existence of another tree T''?

Answer:

Assume T does *not* represent an optimal prefix-free code for C.
Then there exists an optimal tree T'' for C such that $B(T'') < B(T)$.

2.3 (0.2pt)

Applying Lemma 15.2: How can we use Lemma 15.2 to refine our choice of the optimal tree T''?

Answer:

By Lemma 15.2, without loss of generality we may assume T'' has x and y as sibling leaves (i.e., the two least-frequent characters are siblings at maximum depth in T'').

2.4 (0.2pt)

Constructing a Better Tree for C': How can we construct a tree T''' for C' from T''?

Answer:

Let T''' be the tree obtained from T'' by replacing the common parent of x and y with a single leaf z whose frequency is $z.freq = x.freq + y.freq$.
Then T''' is a valid prefix-code tree for C'.
By the cost relationship from part (a) applied to T'':

$$B(T''') = B(T'') - x.freq - y.freq.$$

2.5 (0.2pt)

Contradiction: Show that $B(T''') < B(T')$, and explain why this is a contradiction.

Answer:

Since $B(T'') < B(T)$, and applying the cost relationship to both T'' and T:

$$\begin{aligned} B(T''') &= B(T'') - x.freq - y.freq \\ &< B(T) - x.freq - y.freq \\ &= B(T'). \end{aligned}$$

This means T''' is a prefix-code tree for C' with cost strictly less than $B(T')$, contradicting the assumption that T' is an *optimal* prefix-code tree for C'.
Therefore, T must represent an optimal prefix-free code for C.

3 Transpose of a Graph (1pt)

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. That is, G^T is G with all its edges reversed.

We want to compute G^T from G.

3.1 (0.4pt)

Adjacency List (Pseudocode): Given the adjacency-list representation of G, describe an efficient algorithm to compute G^T. Write the pseudocode below.

Answer:

```
procedure TRANSPOSE-ADJ-LIST(G)
    Let G^T.V = G.V
    Let G^T.Adj be a new array of empty lists
    for each u ∈ G.V do
        for each v ∈ G.Adj[u] do
            Append u to G^T.Adj[v]
        end for
    end for
    return G^T
end procedure
```

3.2 (0.1pt)

Adjacency List (Runtime): Analyze the running time of your algorithm for the adjacency-list representation.

Answer:

$$\boxed{O(V+E)}$$

3.3 (0.4pt)

Adjacency Matrix (Pseudocode): Given the adjacency-matrix representation of G, describe an efficient algorithm to compute G^T. Write the pseudocode below.

Answer:

```
procedure TRANSPOSE-ADJ-MATRIX(A)
    Let n = A.rows
    Let A^T be a new n × n matrix
    for i = 1 to n do
        for j = 1 to n do
            A^T[j][i] = A[i][j]
        end for
    end for
    return A^T
end procedure
```

3.4 (0.1pt)

Adjacency Matrix (Runtime): Analyze the running time of your algorithm for the adjacency-matrix representation.

Answer:

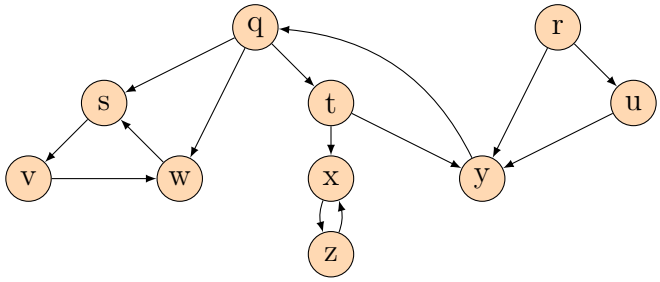
$$\boxed{O(V^2)}$$

4 Strongly Connected Components (1pt)

Consider the procedure `STRONGLY-CONNECTED-COMPONENTS(G)` given below:

1. call `DFS(G)` to compute finish times $u.f$ for each vertex u
2. compute G^T
3. call `DFS`(G^T), but in the main loop of `DFS`, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
4. output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Run this procedure on the graph shown below.



Assume that the main loop of `DFS` (lines 5-7) considers vertices in alphabetical order $(q, r, s, t, u, v, w, x, y, z)$ and that the adjacency lists are also sorted alphabetically.

4.1 (0.5pt)

List the finish time $u.f$ for each vertex u computed in line 1.

💡 **Format:** Write your answer as a list of `vertex: time` pairs, separated by commas. Sort the vertices alphabetically. **Example:** `a: 1, b: 2, c: 3`

Answer:

$$\boxed{q : 16, r : 20, s : 7, t : 15, u : 19, v : 6, w : 5, x : 12, y : 14, z : 11}$$

4.2 (0.5pt)

List the strongly connected components produced by the algorithm.

💡 **Format:** Write each component as a set of vertices enclosed in braces. List the components in the order they are discovered.
Example: `{a, b}, {c}, {d, e, f}`

Answer:

$$\boxed{\{r\}, \{u\}, \{q, t, y\}, \{x, z\}, \{s, v, w\}}$$

5 Minimum Spanning Tree (1pt)

💡 This problem is adapted from Exercise 21.1-3 in *Introduction to Algorithms* 4th edition.

Show that if an edge (u, v) is contained in some minimum spanning tree, then it is a light edge crossing some cut of the graph.

Answer:

Proof. Let T be an MST containing edge (u, v). We must exhibit a cut that (u, v) crosses as a light edge.
Remove (u, v) from T; this splits T into two subtrees T_u (containing u) and T_v (containing v). Let $S = V(T_u)$ (the vertices in u's subtree). The partition $(S, V \setminus S)$ defines a cut, and (u, v) crosses this cut.
(u, v) **is light for this cut.** Suppose for contradiction that some other edge (x, y) crosses the same cut with $w(x, y) < w(u, v)$. Replacing (u, v) with (x, y) in T yields a spanning tree T' with $w(T') = w(T) - w(u, v) + w(x, y) < w(T)$, contradicting the minimality of T. Therefore (u, v) is a light edge crossing the cut $(S, V \setminus S)$. $\square$

6 Prim's Algorithm (1 pt)

💡 This problem is adapted from Exercise 21.2-2 in *Introduction to Algorithms* 4th edition.

🔧 Min-Priority Queue

A **min-priority** queue
(with a **binary min-heap** implementation)
is a data structure that
provides the following operations —

- BUILD-QUEUE(V) — build a queue in $O(|V|)$ time.
- EXTRACT-MIN(Q) — remove and return the element with the lowest `key` in $O(\log |V|)$ time.
- DECREASE-KEY($Q, u,$ `newKey`) — reduce the `key` of u to `newKey` in $O(\log |V|)$ time.

💡 The min-priority queue helps us to find the light edge to add.

Give a simple implementation of Prim's algorithm that runs in $O(V^2)$ time when the graph $G = (V, E)$ is represented as an adjacency matrix.

6.1 (0.6 pt)

Complete the following pseudocode by filling in the body of the `while` loop.

```
Prim-Matrix(G, w, r)
let B[1..|V|] be a new bit array
for each u ∈ G.V do
    u.key = ∞
    u.π = NIL
    B[u] = True
end for
r.key = 0
Q = G.V
while Q ≠ ∅ do
                                        ▷ Fill in the missing part here
end while
```

Answer:

Solution 1 (Linear scan — no min-priority queue):
We do *not* use the min-priority queue. Instead, we find the minimum-key vertex by a simple **linear scan** over the bit array B.

```
while Q ≠ ∅ do
    u = NIL
    for each v ∈ G.V do
        if B[v] == True and (u == NIL or v.key < u.key) then
            u = v
        end if
    end for
    B[u] = False
```

```
    for each v ∈ G.V do
        if B[v] == True and G.A[u, v] == 1 and w(u, v) < v.key then
            v.π = u
            v.key = w(u, v)
        end if
    end for
end while
```

Solution 2 (Using min-priority queue):
We can also use the min-priority queue, but this does *not* change the $O(V^2)$ complexity because the inner loop already scans all V vertices via the adjacency matrix.

```
while Q ≠ ∅ do
    u = EXTRACT-MIN(Q)
    B[u] = False
    for each v ∈ G.V do
        if B[v] == True and G.A[u, v] == 1 and w(u, v) < v.key then
            v.π = u
            v.key = w(u, v)
            DECREASE-KEY(Q, v, w(u, v))
        end if
    end for
end while
```

6.2 (0.4 pt)

Explain why the implementation above takes $O(V^2)$ time.

Answer:

Solution 1: The linear scan to find the minimum-key vertex takes $O(V)$. The inner loop scans all V vertices via the adjacency matrix in $O(V)$. Each iteration takes $O(V) + O(V) = O(V)$, and the loop runs $|V|$ times, giving $O(V^2)$.
Solution 2: Using a binary min-heap, EXTRACT-MIN takes $O(\log V)$ per call, totaling $O(V \log V)$. However, the inner loop still scans all V vertices in the adjacency matrix row, so each iteration takes $O(\log V) + O(V) = O(V)$. The total is still $O(V^2)$.
In both cases, the $O(V^2)$ bottleneck comes from scanning adjacency matrix rows, not from the extract-min step.

7 Bellman-Ford Algorithm (1pt)

Bellman-Ford's Algorithm

```
Bellman-Ford(G, w, s)
INIT-SINGLE-SOURCE(G, s)
for i = 1 to |G.V| − 1 do
    for each edge (u, v) ∈ G.E do
        RELAX(u, v, w)
    end for
end for
for each edge (u, v) ∈ G.E do
    if v.d > u.d + w(u, v) then
        return False                    ▷ There is a negative cycle
    end if
end for
return True
```

This problem is adapted from Exercise 21.1-1 in *Introduction to Algorithms* 4th edition.

Consider the directed graph defined by the adjacency list and weights below. Run the Bellman-Ford algorithm on this graph using z as the source vertex.

Relax edges in the following order:

(z, a), (z, d), (a, c), (a, d), (b, z), (b, d), (c, a), (d, b)

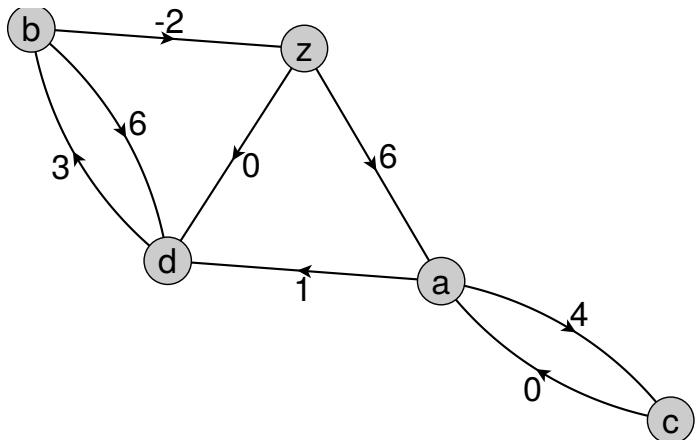


Figure 2: The Directed Graph

7.1 (0.4 pt)

Fill in the d values (shortest path estimates) after each round of relaxation. Initialization is Round 0.

Intended interpretation ("round" = one full pass over all edges):

Round	z	a	b	c	d
0	0	∞	∞	∞	∞
1	0	6	3	10	0
2	0	6	3	10	0
3	0	6	3	10	0
4	0	6	3	10	0

Alternative interpretation ("round" = each individual edge relaxation):

Step	z	a	b	c	d
Init	0	∞	∞	∞	∞
R1: (z,a)	0	6	∞	∞	∞
R1: (z,d)	0	6	∞	∞	0
R1: (a,c)	0	6	∞	10	0
R1: (a,d)	0	6	∞	10	0
R1: (b,z)	0	6	∞	10	0
R1: (b,d)	0	6	∞	10	0
R1: (c,a)	0	6	∞	10	0
R1: (d,b)	0	6	3	10	0
R2: (z,a)	0	6	3	10	0
R2: (z,d)	0	6	3	10	0
R2: (a,c)	0	6	3	10	0
R2: (a,d)	0	6	3	10	0
R2: (b,z)	0	6	3	10	0
R2: (b,d)	0	6	3	10	0
R2: (c,a)	0	6	3	10	0
R2: (d,b)	0	6	3	10	0
R3: (z,a)	0	6	3	10	0
R3: (z,d)	0	6	3	10	0
R3: (a,c)	0	6	3	10	0
R3: (a,d)	0	6	3	10	0
R3: (b,z)	0	6	3	10	0
R3: (b,d)	0	6	3	10	0
R3: (c,a)	0	6	3	10	0
R3: (d,b)	0	6	3	10	0
R4: (z,a)	0	6	3	10	0
R4: (z,d)	0	6	3	10	0
R4: (a,c)	0	6	3	10	0
R4: (a,d)	0	6	3	10	0
R4: (b,z)	0	6	3	10	0
R4: (b,d)	0	6	3	10	0
R4: (c,a)	0	6	3	10	0
R4: (d,b)	0	6	3	10	0

Note: The alternative interpretation is accepted for *this quiz only.* On the final exam, "round" will strictly mean one complete pass over all edges (the outer loop), and points will be deducted for the inner-loop interpretation.

7.2 (0.4 pt)

Fill in the π values (predecessors) after each round of relaxation. Use NIL for no predecessor.

Intended interpretation ("round" = one full pass over all edges):

Round	z	a	b	c	d
0	NIL	NIL	NIL	NIL	NIL
1	NIL	z	d	a	z
2	NIL	z	d	a	z
3	NIL	z	d	a	z
4	NIL	z	d	a	z

Alternative interpretation ("round" = each individual edge relaxation):

Step	z	a	b	c	d
Init	NIL	NIL	NIL	NIL	NIL
R1: (z,a)	NIL	z	NIL	NIL	NIL
R1: (z,d)	NIL	z	NIL	NIL	z
R1: (a,c)	NIL	z	NIL	a	z
R1: (a,d)	NIL	z	NIL	a	z
R1: (b,z)	NIL	z	NIL	a	z
R1: (b,d)	NIL	z	NIL	a	z
R1: (c,a)	NIL	z	NIL	a	z
R1: (d,b)	NIL	z	d	a	z
R2: (z,a)	NIL	z	d	a	z
R2: (z,d)	NIL	z	d	a	z
R2: (a,c)	NIL	z	d	a	z
R2: (a,d)	NIL	z	d	a	z
R2: (b,z)	NIL	z	d	a	z
R2: (b,d)	NIL	z	d	a	z
R2: (c,a)	NIL	z	d	a	z
R2: (d,b)	NIL	z	d	a	z
R3: (z,a)	NIL	z	d	a	z
R3: (z,d)	NIL	z	d	a	z
R3: (a,c)	NIL	z	d	a	z
R3: (a,d)	NIL	z	d	a	z
R3: (b,z)	NIL	z	d	a	z
R3: (b,d)	NIL	z	d	a	z
R3: (c,a)	NIL	z	d	a	z
R3: (d,b)	NIL	z	d	a	z
R4: (z,a)	NIL	z	d	a	z
R4: (z,d)	NIL	z	d	a	z
R4: (a,c)	NIL	z	d	a	z
R4: (a,d)	NIL	z	d	a	z
R4: (b,z)	NIL	z	d	a	z
R4: (b,d)	NIL	z	d	a	z
R4: (c,a)	NIL	z	d	a	z
R4: (d,b)	NIL	z	d	a	z

Note: Same as above—the alternative interpretation is accepted for this quiz only.

7.3 (0.2 pt)

What value does the Bellman-Ford algorithm return?

Answer:

- ☒ True
- ☐ False

8 Dijkstra's Algorithm (1pt)

💡 This problem is adapted from Exercise 24.3-4 in *Introduction to Algorithms* 4th edition.

Consider the directed graph defined by the adjacency list and weights below. Run Dijkstra's algorithm on this graph using z as the source vertex.

Adjacency list (neighbor, weight): listed in alphabetical order by neighbor.

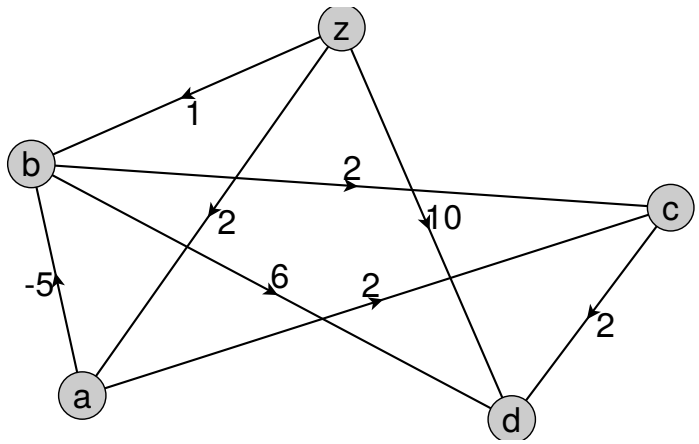


Figure 3: The Directed Graph

8.1 (0.4 pt)

Give the final d values (shortest path estimates) after Dijkstra's algorithm finishes.

	z	a	b	c	d
Final	0	2	-3	3	5

Explanation: Dijkstra extracts vertices in order of their current d values. Since b (with $b.d = 1$) is extracted before a (with $a.d = 2$), the negative edge $a \to b$ with weight -5 updates $b.d$ to -3 only **after** b has already been extracted. Therefore b's outgoing edges are never re-relaxed with the improved estimate.

8.2 (0.4 pt)

Give the final π values (predecessors) after Dijkstra's algorithm finishes. Use NIL for no predecessor.

	z	a	b	c	d
Final	NIL	z	a	b	c

Explanation: The π values record which vertex last relaxed each vertex. Since b is extracted before a, $b.\pi$ is initially z. When a is later extracted, the relaxation $a \to b$ updates $b.d$ and sets $b.\pi = a$. However, the downstream vertices c and d retain their predecessors from the earlier (stale) relaxation through b.

8.3 (0.2 pt)

Does Dijkstra's algorithm produce the correct shortest path weights for this graph?

Answer:

☐ Yes

☒ No **Explanation:** No. A negative-weight edge does not *always* cause Dijkstra to fail — it fails only when the negative edge lets a later-discovered path improve $v.d$ for a vertex v that has **already been extracted**. In this graph, b is extracted (with $b.d = 1$) before a (with $a.d = 2$). When a is later extracted, it relaxes $a \to b$ and sets $b.d = -3$, but b has already left the queue, so its outgoing edges are never re-relaxed. Bellman–Ford gives the correct values: $c.d = -1$ and $d.d = 1$, whereas Dijkstra computes $c.d = 3$ and $d.d = 5$.

⭐🌻🌷✨🌻🌷🐰🐰❤️⭐ **The End of The Quiz** ⭐🌻🌷✨🌻🌷🐰🐰❤️⭐